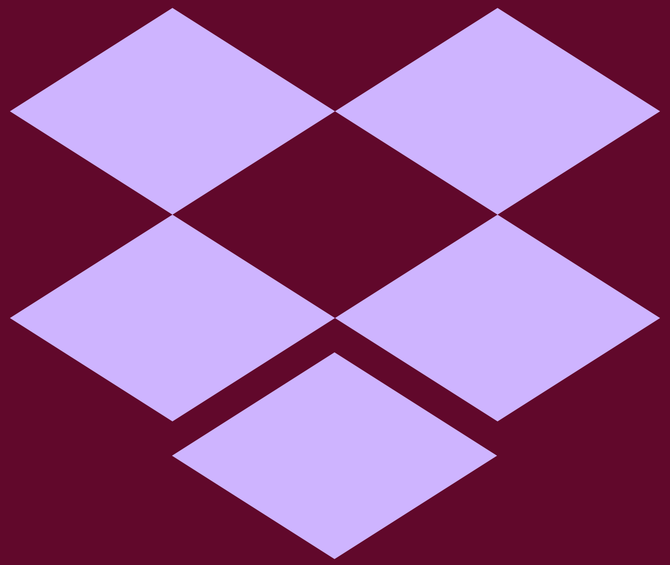
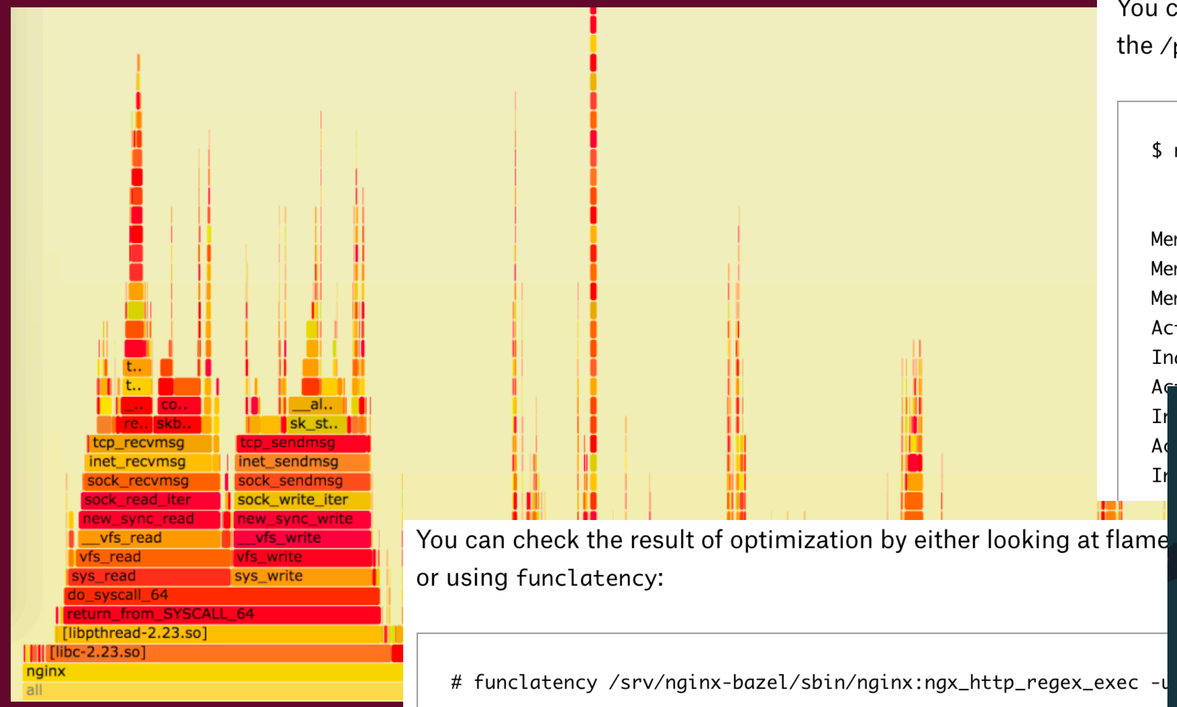
Evaluating BBRv2 On the Edge

Alexey Ivanov, Dropbox

V4.1



Traffic Optimizations

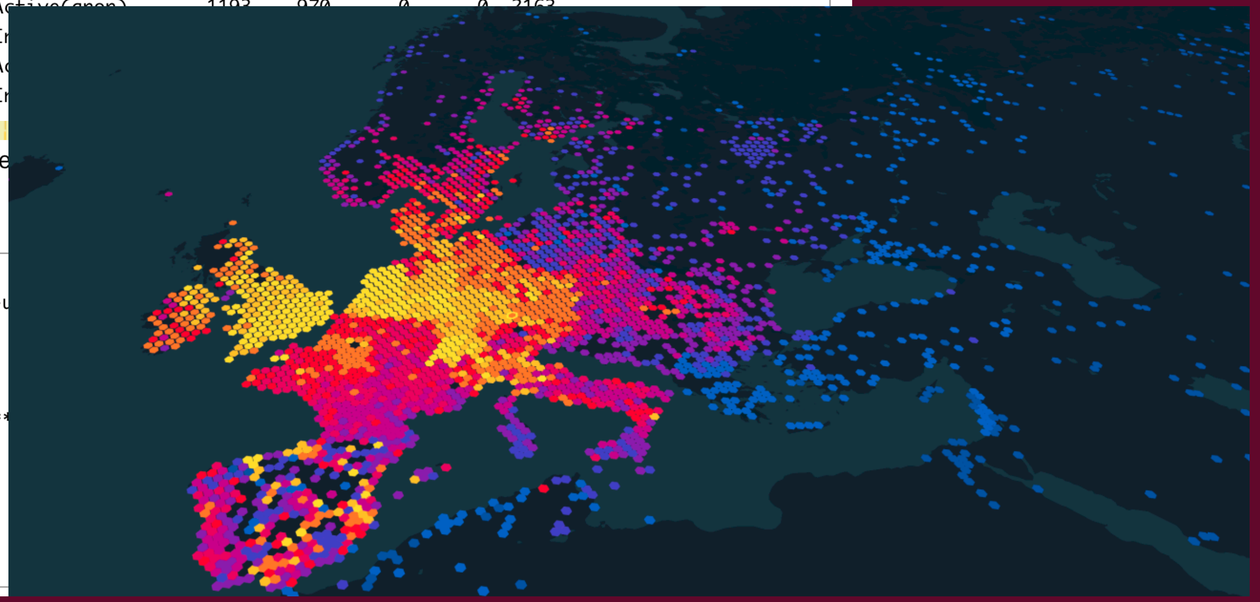


You can also ask `numastat` to output per-node memory usage statistics in the `/proc/meminfo` format:

```
$ numastat -m -c
                 Node 0 Node 1 Node 2 Node 3 Total
                 ------ ------ ------ ------ -----
MemTotal          32150  32214      0      0 64363
MemFree             462   5793      0      0  6255
MemUsed           31688  26421      0      0 58109
Active            16021   8588      0      0 24608
Inactive          13436  16121      0      0 29557
```

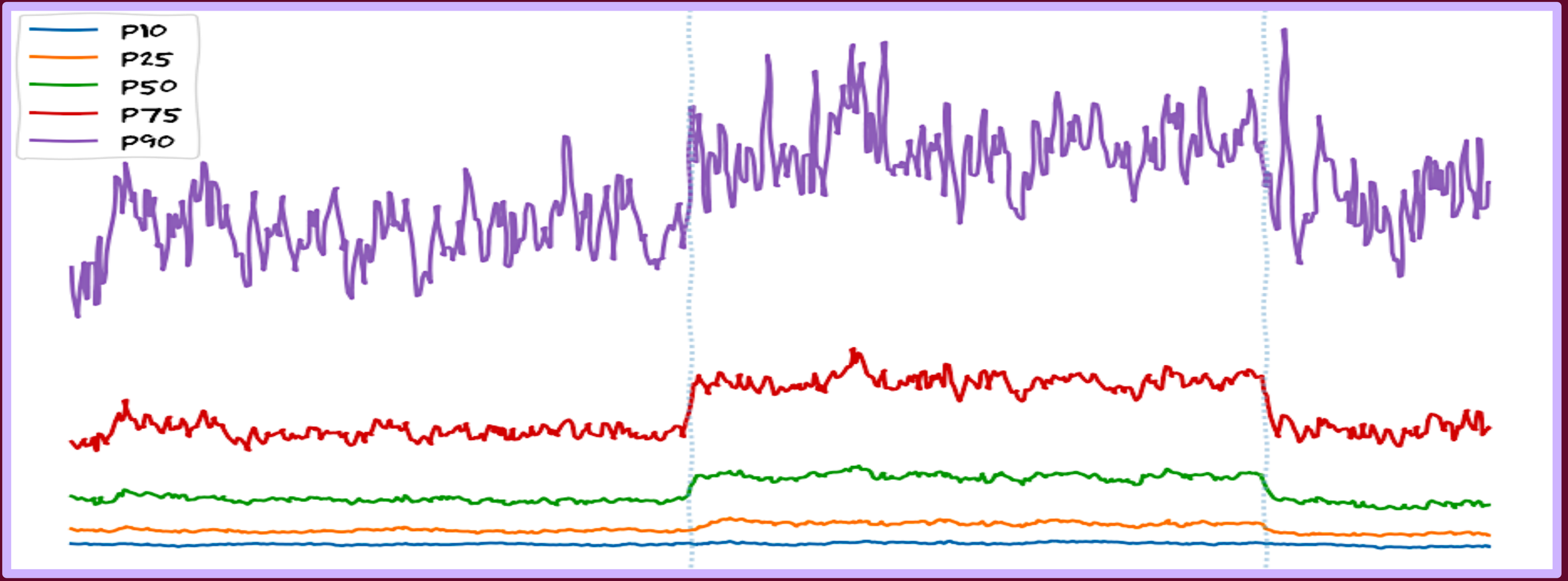
You can check the result of optimization by either looking at flame
or using `funclatency`:

```
# funclatency /srv/nginx-bazel/sbin/nginx:ngx_http_regex_exec -u
...
     usecs               : count     distribution
         0 -> 1          : 1159     |**********
         2 -> 3          : 4468     |****************************
         4 -> 7          : 622      |*****
         8 -> 15         : 610      |*****
        16 -> 31         : 209      |*
        32 -> 63         : 91       |
```

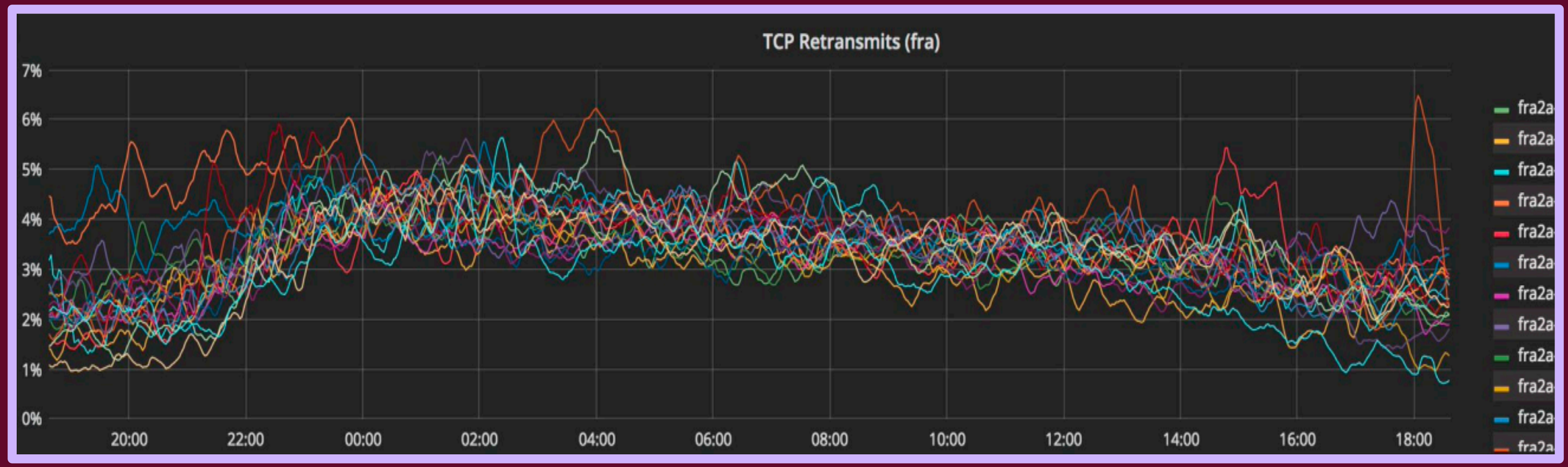


BBRv1

Desktop client's download speed during BBRv1 experiment in 2017.



Initial BBRv1 deployment: some boxes have >6% packet loss.

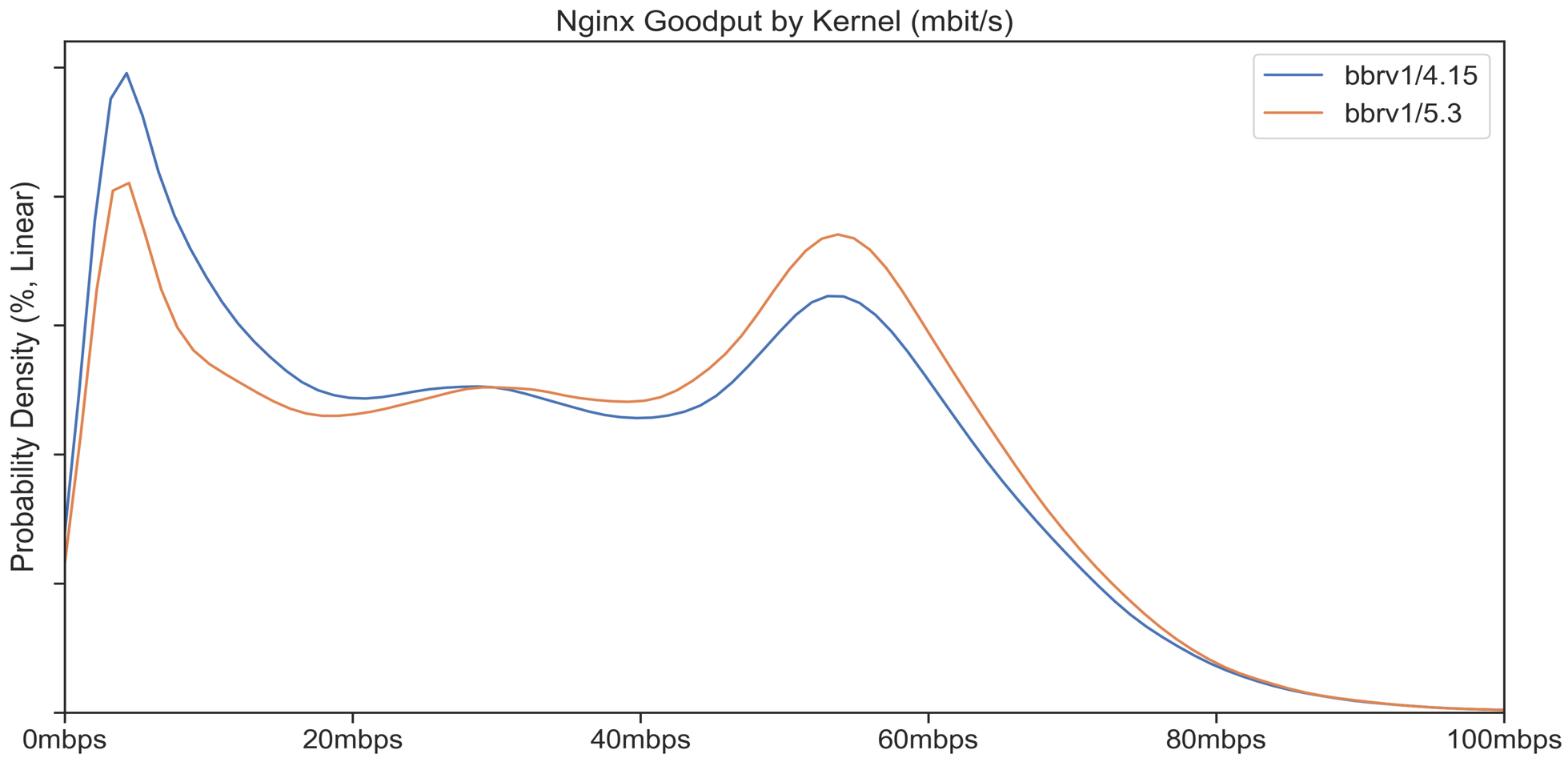


Issues with BBRv1.

- Low throughput for Reno/CUBIC flows sharing a bottleneck with bulk BBR flows
- Loss-agnostic; high packet loss rates if bottleneck queue < 1.5*BDP
- Low throughput for paths with high degrees of aggregation (e.g. wifi)
- Throughput variation due to low cwnd in PROBE_RTT
- ECN-agnostic

Caveats

Upgrade your kernels.



Upgrade your userspace.

```
$ ss -tie
ts sack bbr rto:220 rtt:16.139/10.041 ato:40 mss:1448 cwnd:106
ssthresh:52 bytes_acked:9067087 bytes_received:5775 segs_out:6327
segs_in:551 send 76.1Mbps lastsnd:14536 lastrcv:15584
lastack:14504 pacing_rate 98.5Mbps retrans:0/5 rcv_rtt:16.125
rcv_space:14400
```

Upgrade your userspace.

```
$ ss -tie
ts sack bbr rto:220 rtt:16.139/10.041 ato:40 mss:1448 pmtu:1500
rcvmss:1269 advmss:1428 cwnd:106 ssthresh:52 bytes_sent:9070462
bytes_retrans:3375 bytes_acked:9067087 bytes_received:5775
segs_out:6327 segs_in:551 data_segs_out:6315 data_segs_in:12
bbr:(bw:99.5Mbps,mrtt:1.912,pacing_gain:1,cwnd_gain:2) send
76.1Mbps lastsnd:9896 lastrcv:10944 lastack:9864 pacing_rate
98.5Mbps delivery_rate 27.9Mbps delivered:6316 busy:3020ms
rwnd_limited:2072ms(68.6%) retrans:0/5 dsack_dups:5
rcv_rtt:16.125 rcv_space:14400 rcv_ssthresh:65535 minrtt:1.907
```

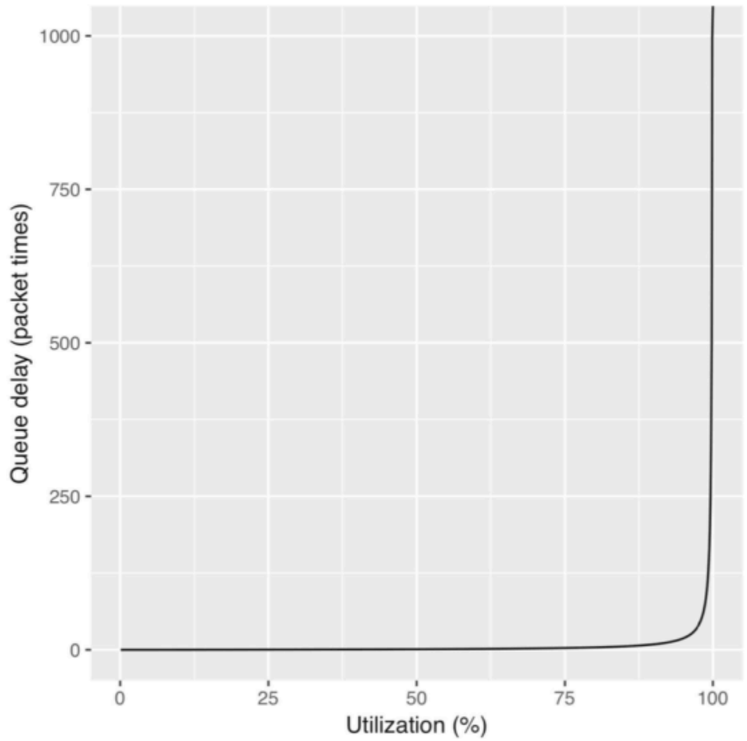
Use fq scheduler.

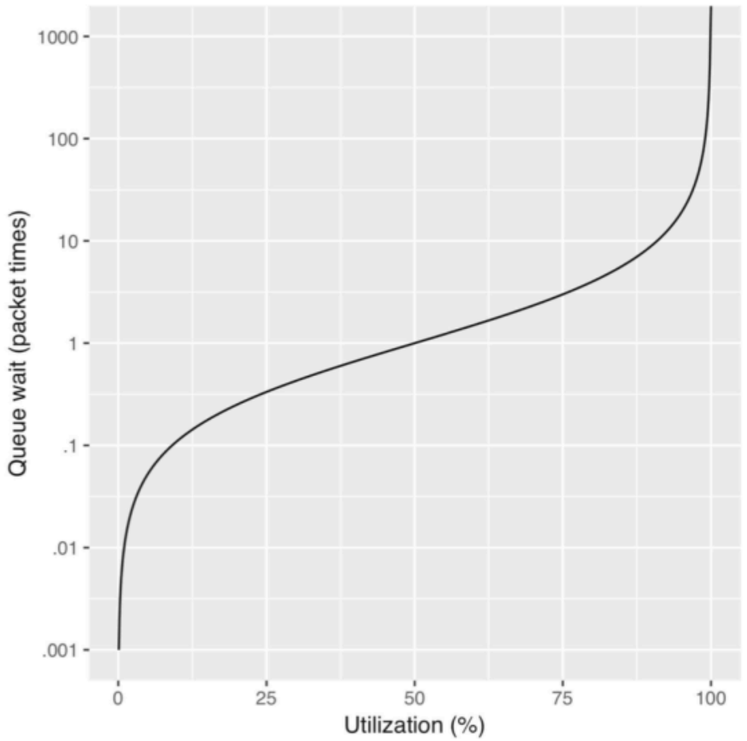
```
$ tc -s qdisc show dev eth0
qdisc mq 1: root
  Sent 100800259362 bytes 81191255 pkt (dropped 122, overlimits 0
requeues 35) backlog 499933b 124p requeues 35
qdisc fq 9cd7: parent 1:17 limit 10000p flow_limit 100p buckets
1024 orphan_mask 1023 quantum 3028 initial_quantum 15140
low_rate_threshold 550Kbit refill_delay 40.0ms
  Sent 1016286523 bytes 806982 pkt (dropped 0, overlimits 0
requeues 0) backlog 0b 0p requeues 0
...
```

Beyond As Fast As Possible

… but AFAP makes bottleneck run at 100%

Queuing theory says this is fragile. E.g., for M/D/1:





9

"Evolving from AFAP – Teaching NICs about time" by Van Jacobson

Disclaimers & Test setup

Specifics of this evaluation.

- Not a low-latency experiment:

  We'll be looking only at "bulk-flows."

- Heavily aggregated data:

  No single flow tcpdump/tcptrace drilldowns.

- Not a lab test:

  Real traffic with all its imperfections.

Test setup.

- A single PoP in Tokyo
- 4 boxes
- Only “bulk” connections (w/ >1Mbyte transferred)
- `ss` sampling and `nginx` log processing

Test setup (cont'd.)

- ~~4.15 kernel, bbr1~~
- 5.3 kernel, cubic
- 5.3 kernel, bbr1
- 5.3 kernel, bbr2

BBRv2 Theory

BBR design principles

- **Leave headroom: leave space for entering flows to grab**
- **React quickly: using loss/ECN, adapt to delivery process *now* to maintain flow balance**
- Don't overreact: don't do a multiplicative decrease on every round trip with loss/ECN
- **Probe deferentially: probe on a time scale to allow coexistence with Reno/CUBIC**
- Probe robustly: try to probe beyond estimated max bw, max volume before we cut est.
- **Avoid overshooting: start probing at an inflight measured to be tolerable**
- Grow scalably: **start probing at 1 extra packet**; grow exponentially to use free capacity

(**bold** = new in v2)

What's new in BBRv2

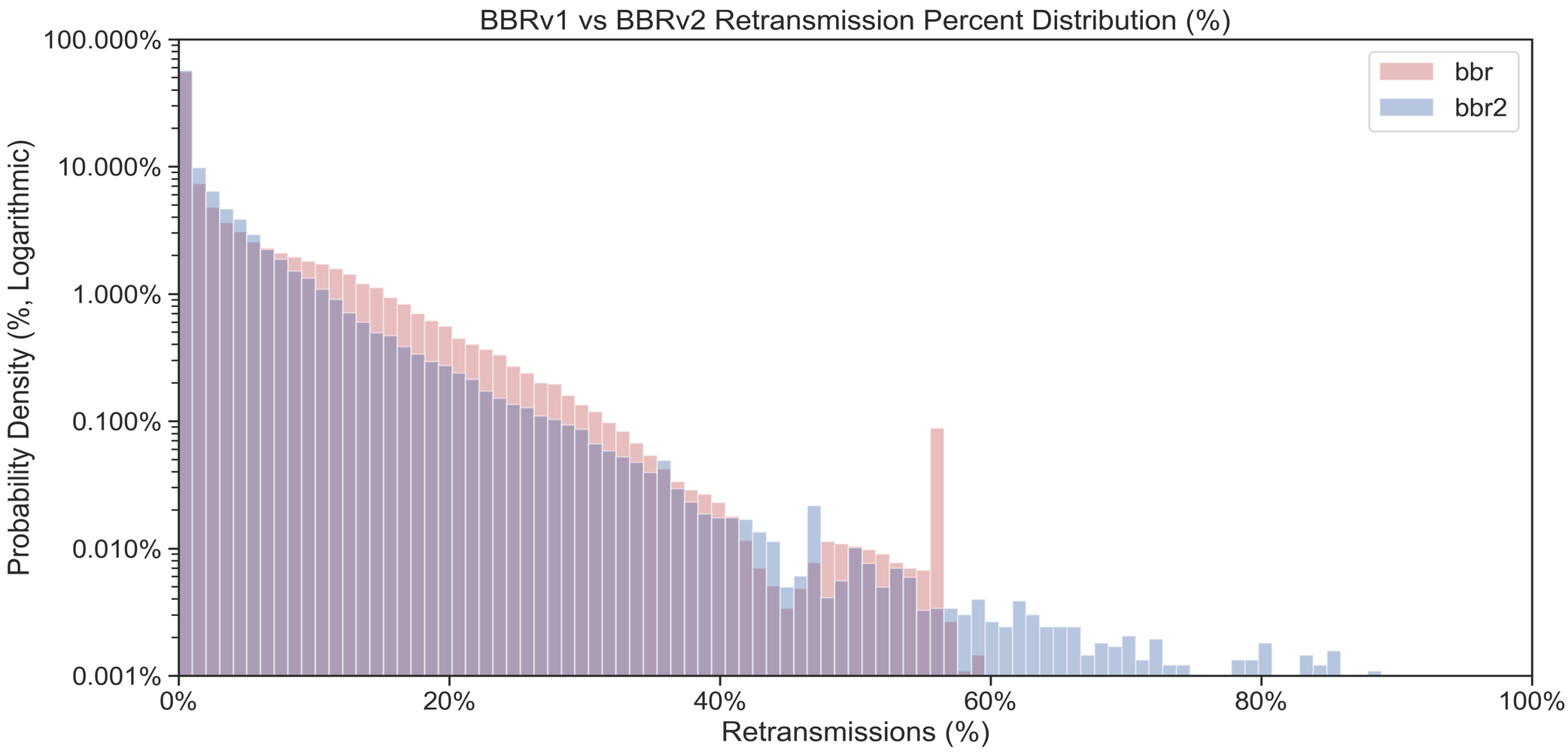
	CUBIC	BBR v1	BBR v2
Model parameters to the state machine	N/A	Throughput, RTT	Throughput, RTT, max aggregation, max inflight
Loss	Reduce cwnd by 30% on window with any loss	N/A	Explicit loss rate target
ECN	RFC3168 (Classic ECN)	N/A	DCTCP-inspired ECN
Startup	Slow-start until RTT rises (Hystart) or any loss	Slow-start until tput plateaus	Slow-start until tput plateaus or ECN/loss rate > target

BBRv2 Properties in Practice

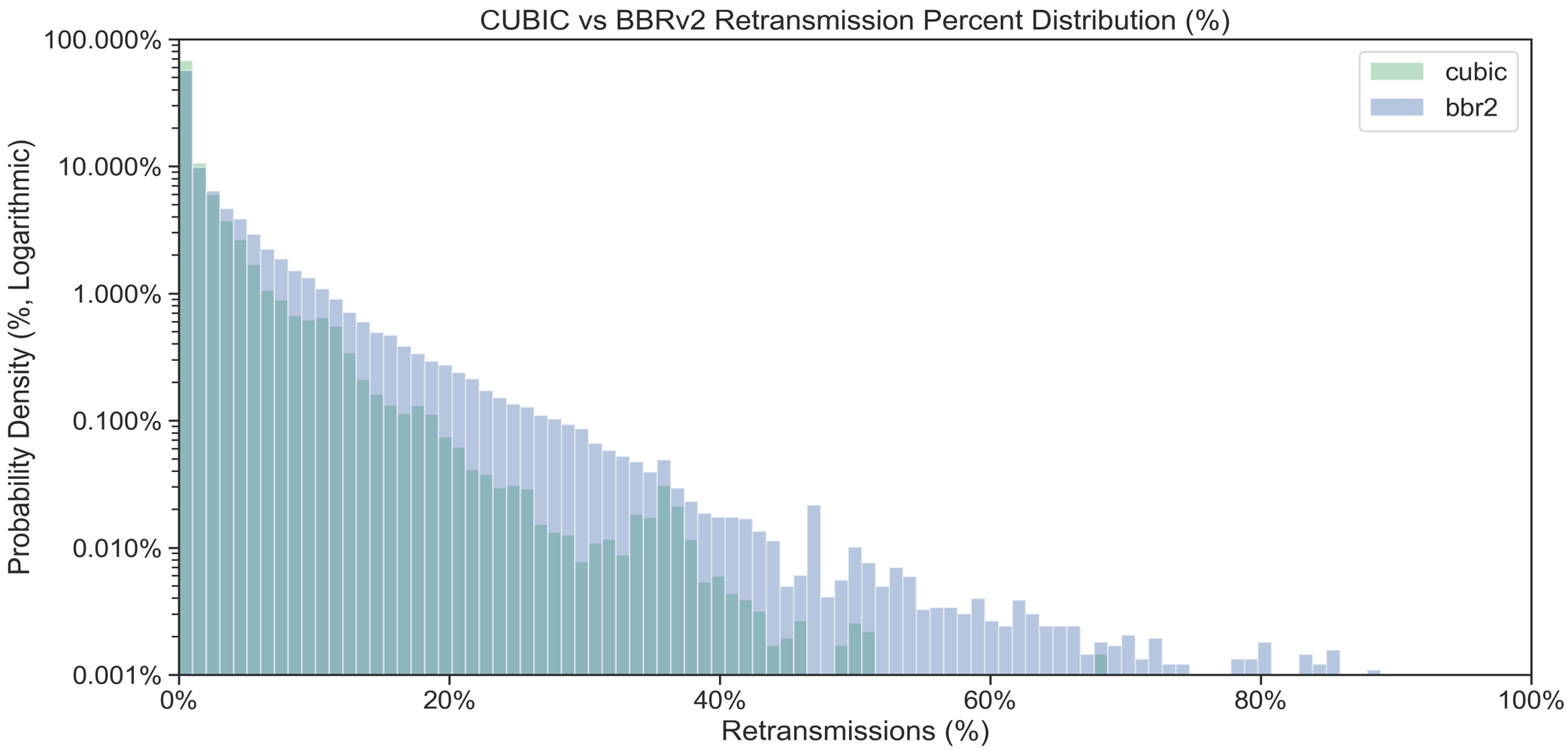
Lower packet loss.



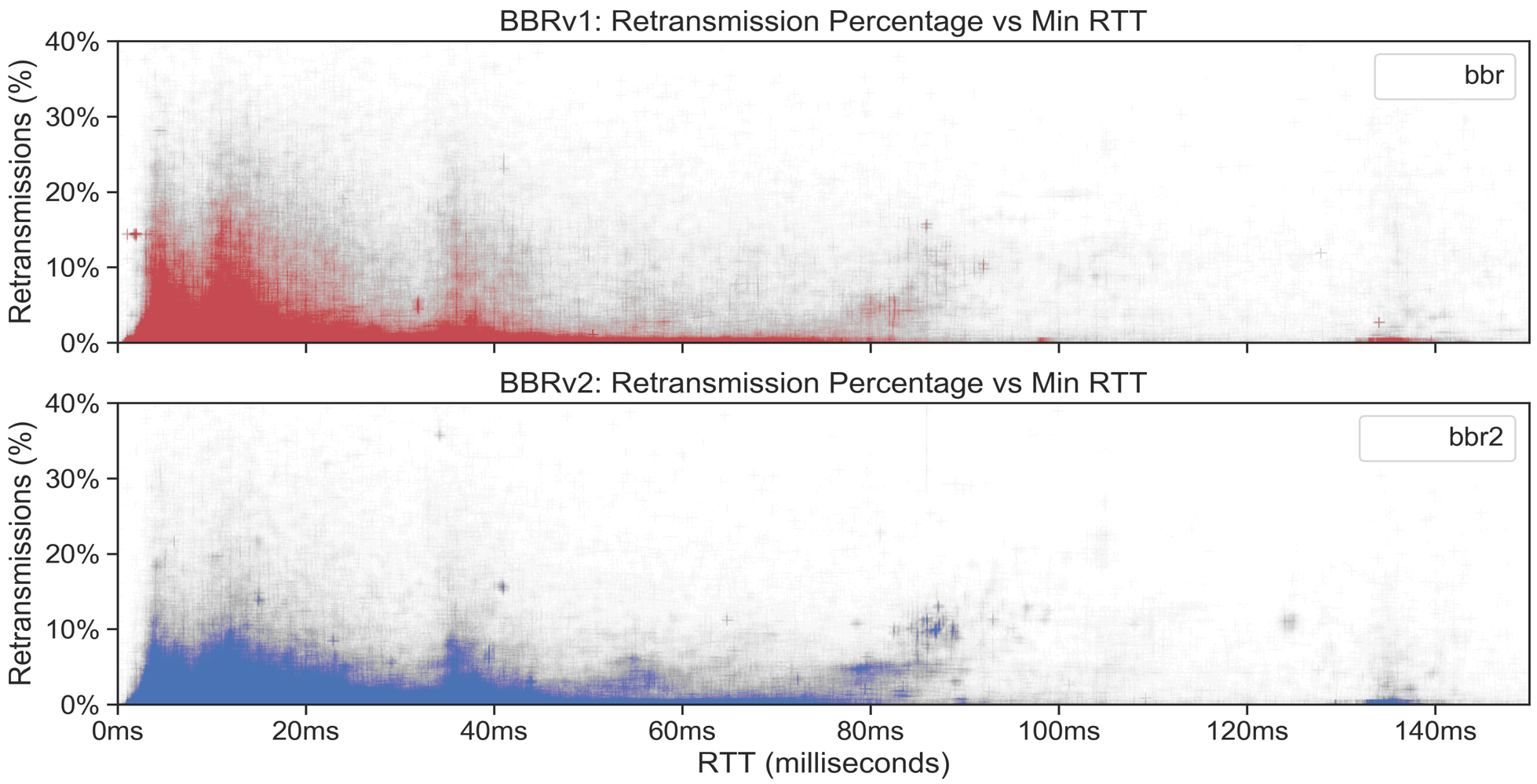
Lower packet loss (vs BBRv1.)



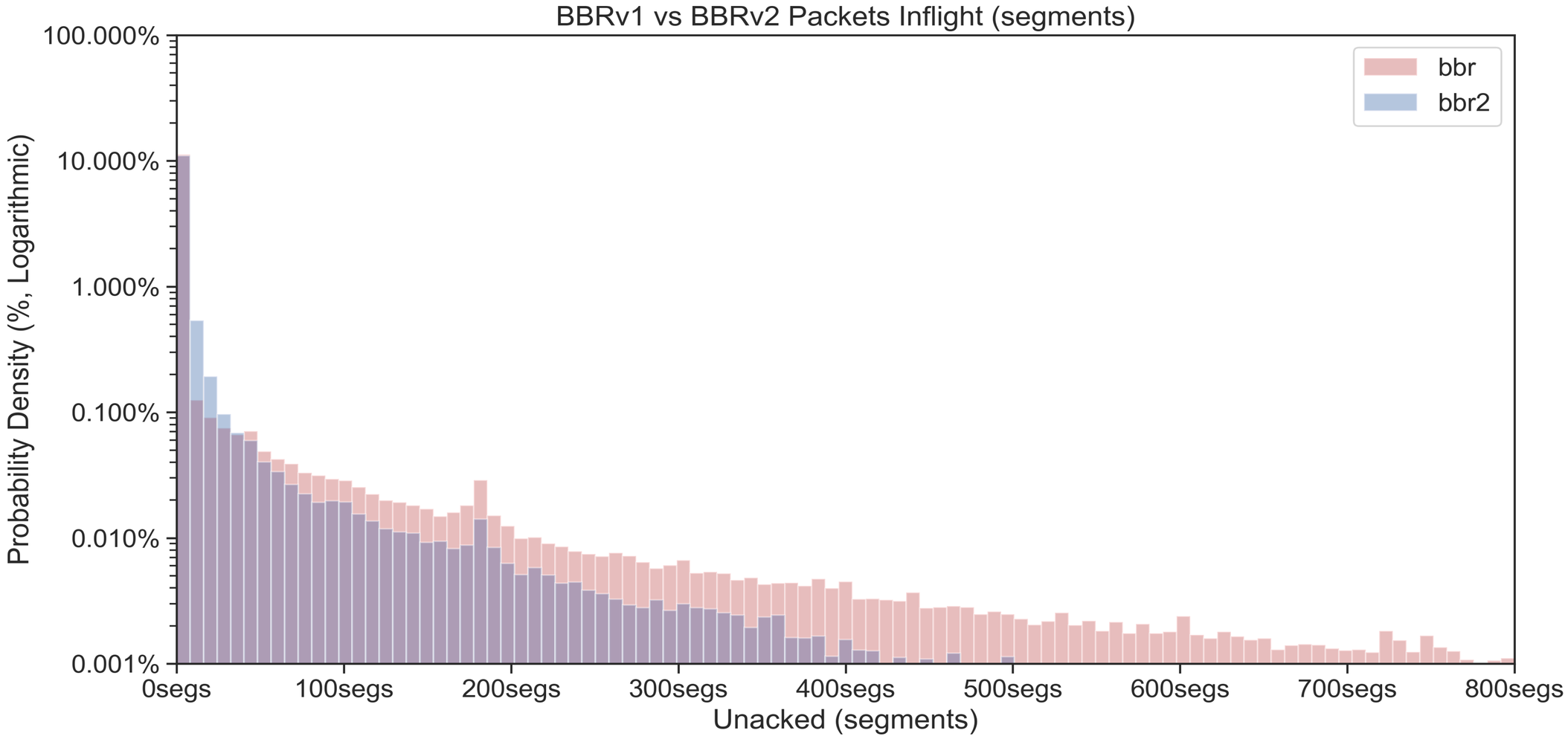
Higher packet loss (vs Cubic.)



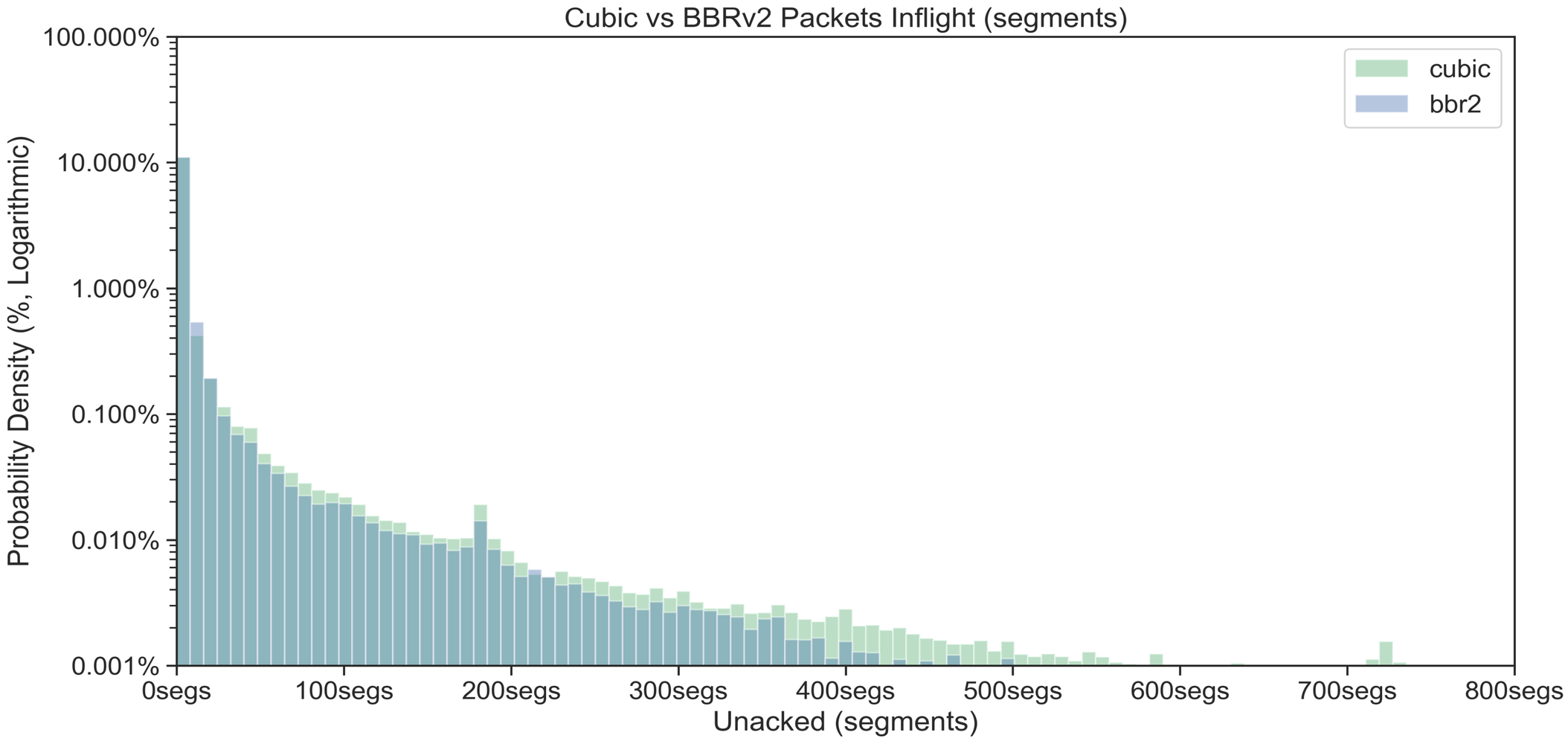
Packet loss vs MinRTT.



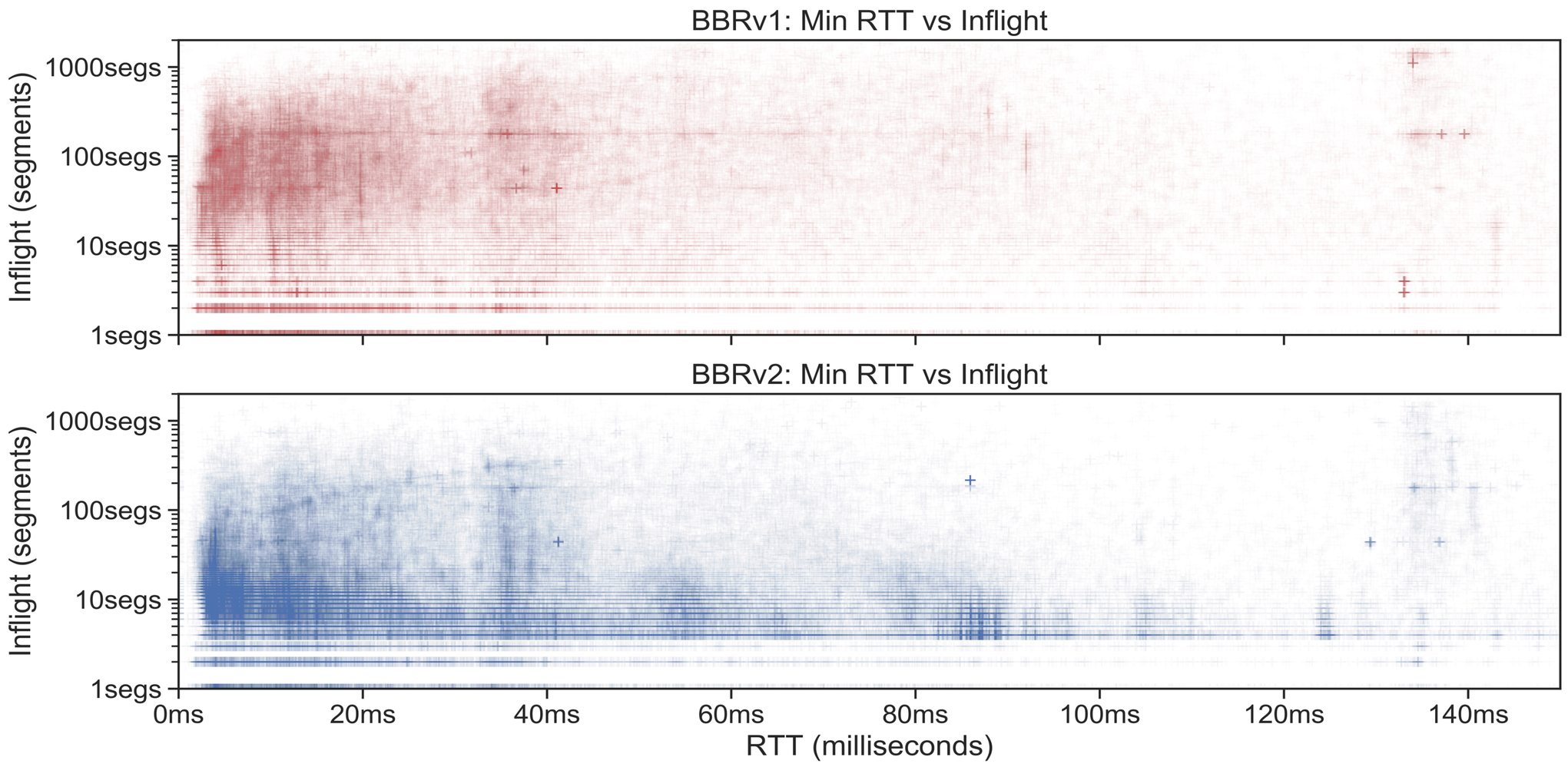
Lower packets inflight (vs BBRv1.)



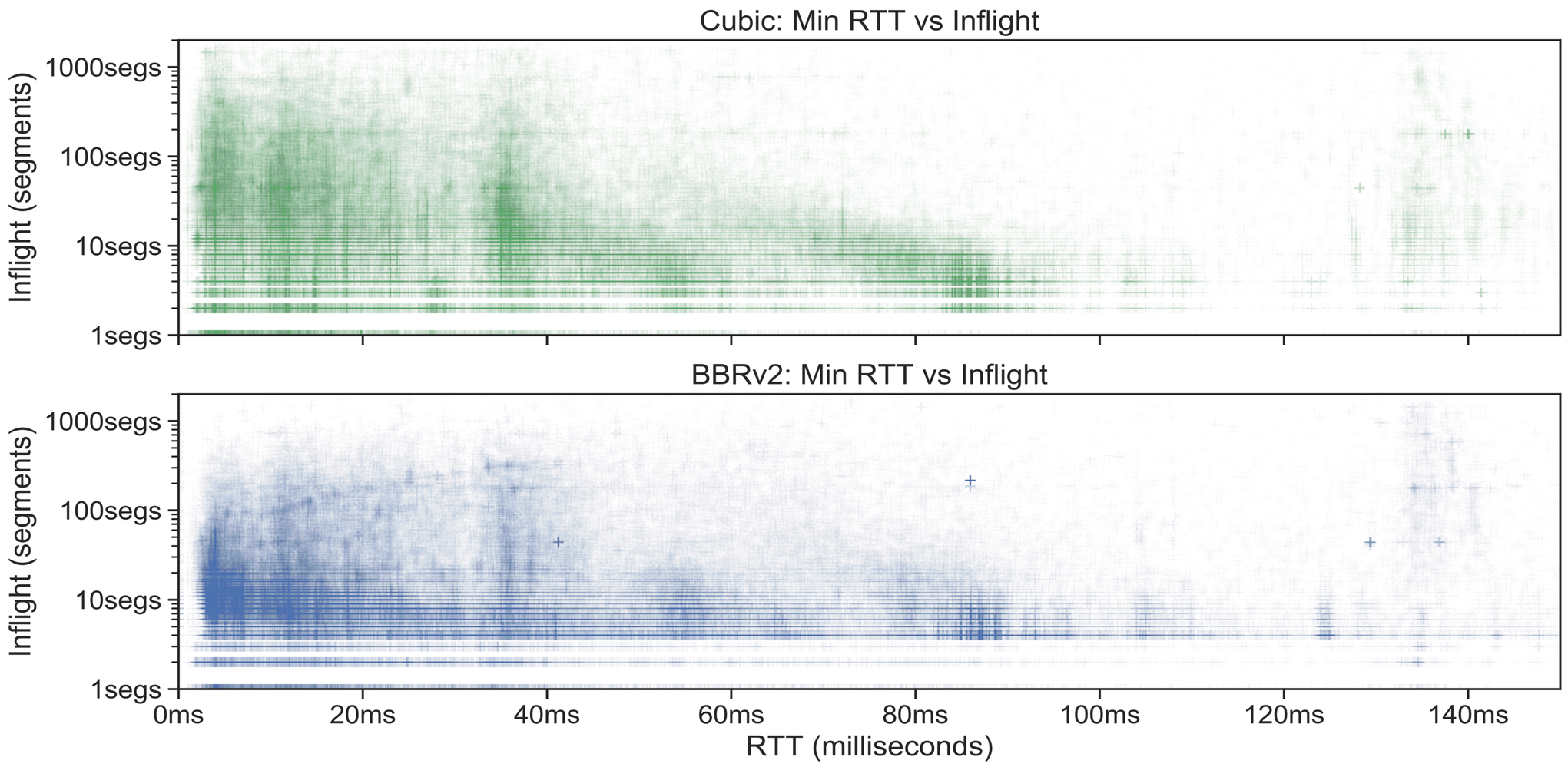
Lower packets inflight (vs Cubic.)



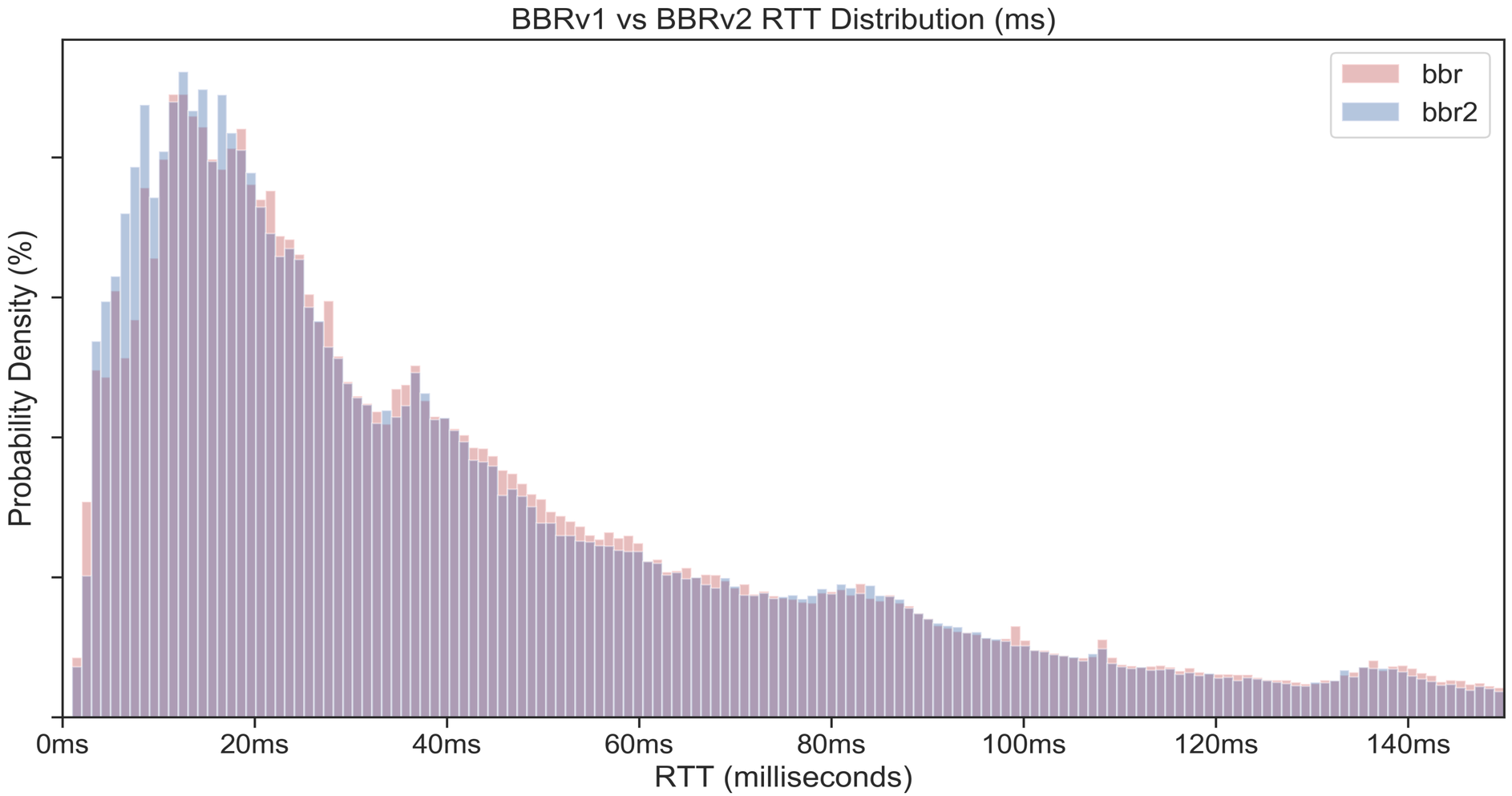
Packets inflight vs mRTT.



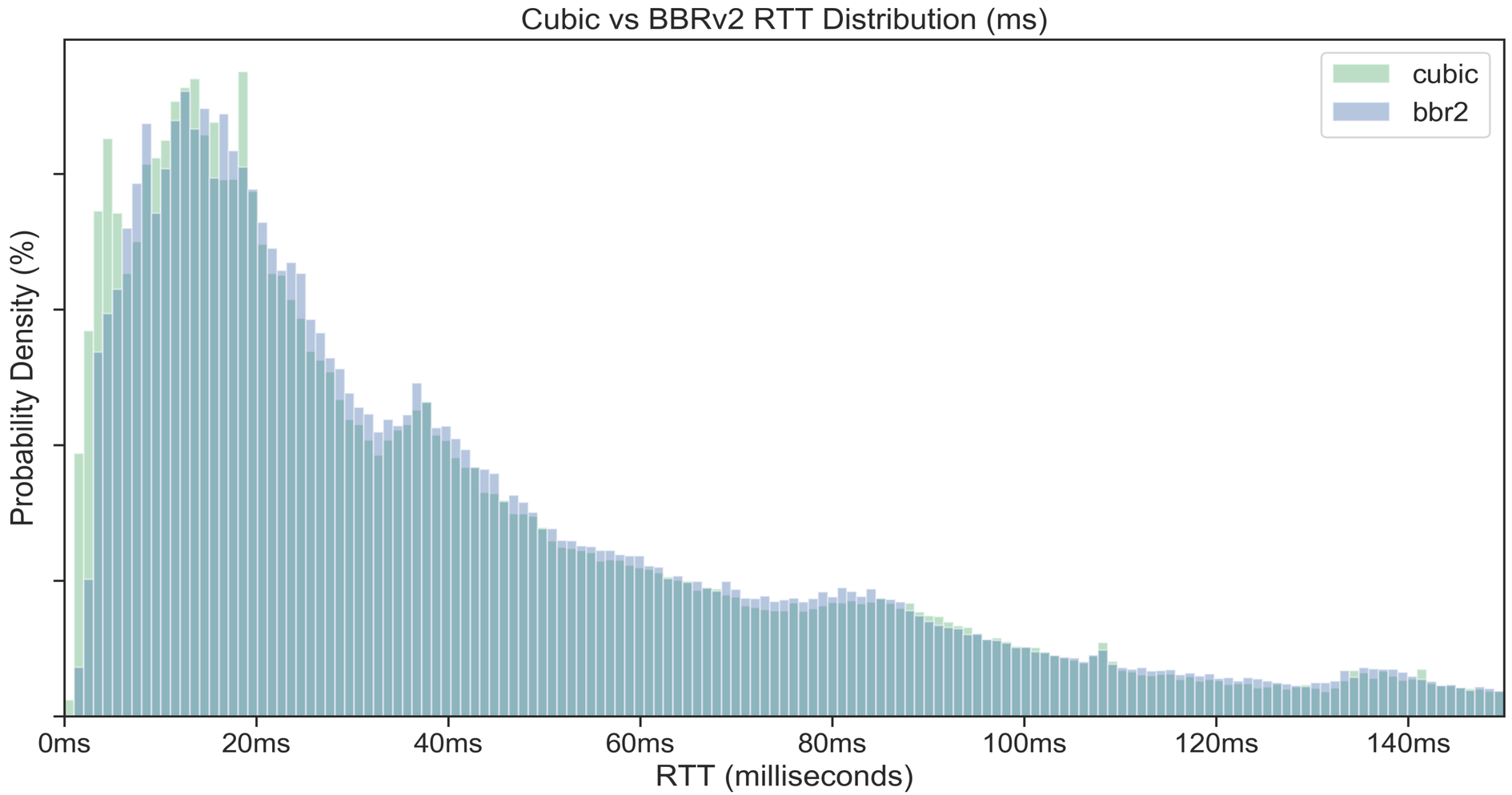
Packets inflight vs mRTT (cont'd).



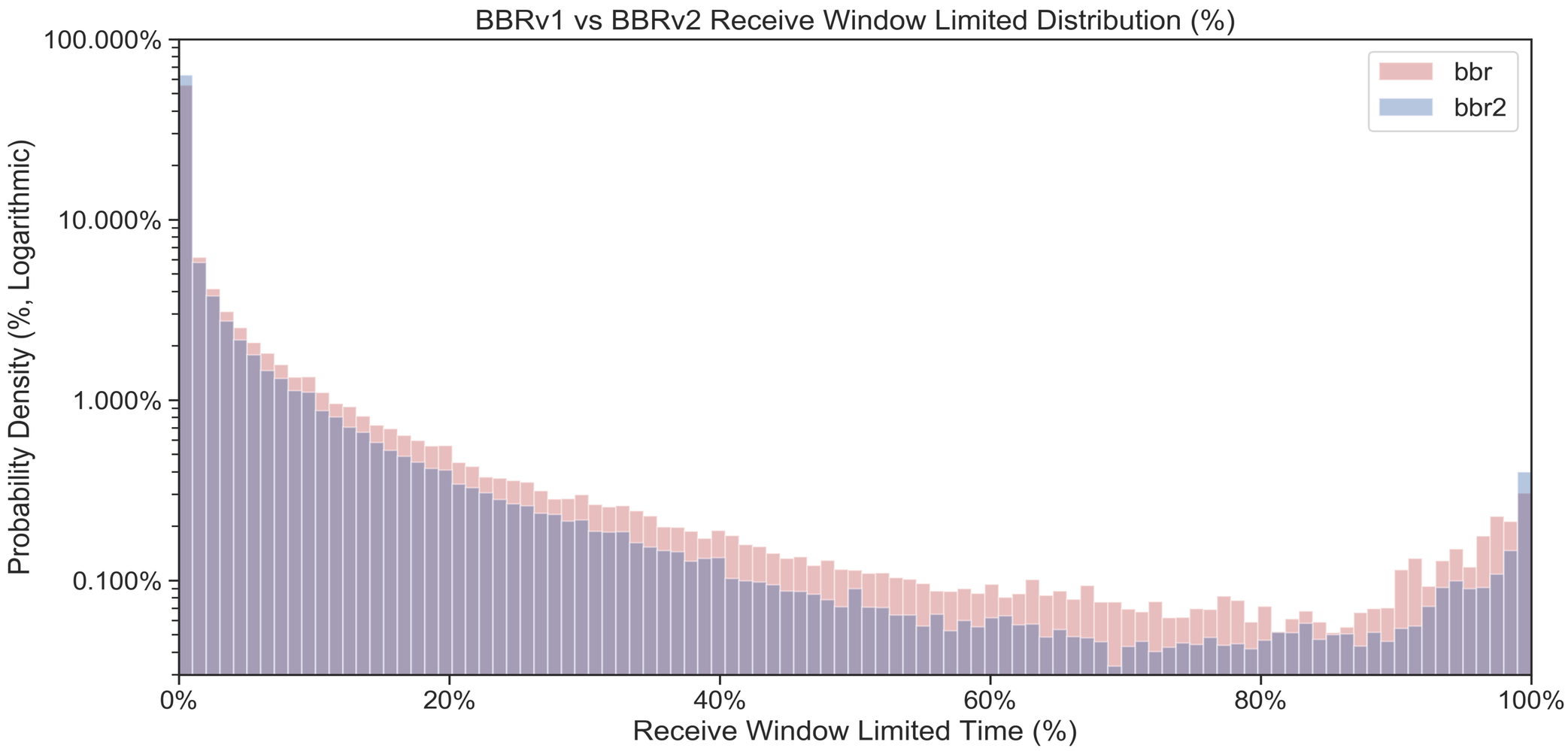
RTT (vs BBRv1.)


BBRv1 vs BBRv2 RTT Distribution (ms)
bbr
bbr2
Probability Density (%)
0ms
20ms
40ms
60ms
80ms
100ms
120ms
140ms
RTT (milliseconds)

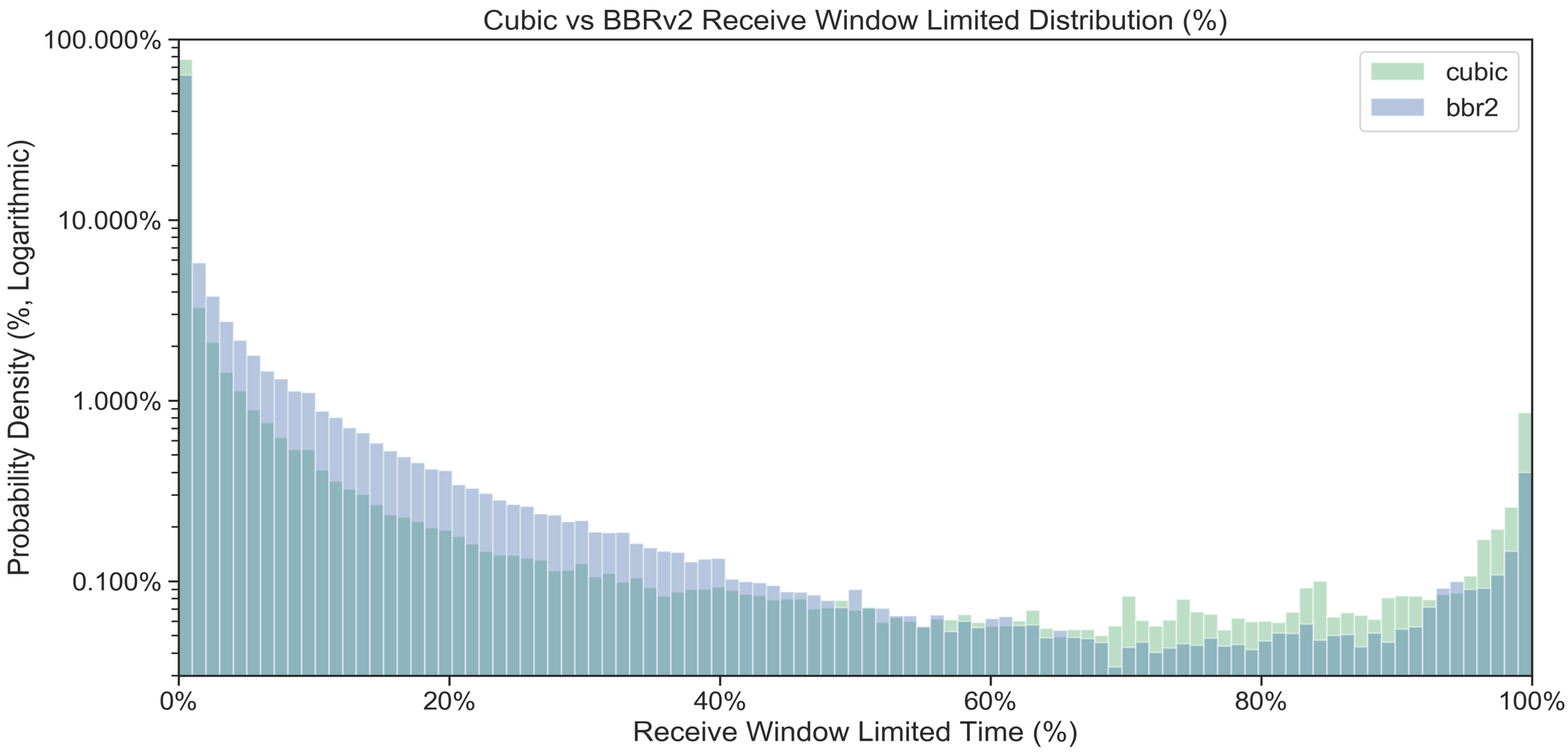
RTT (vs Cubic.)


Cubic vs BBRv2 RTT Distribution (ms)
cubic
bbr2
Probability Density (%)
0ms
20ms
40ms
60ms
80ms
100ms
120ms
140ms
RTT (milliseconds)

RWND-limited (vs BBRv1.)

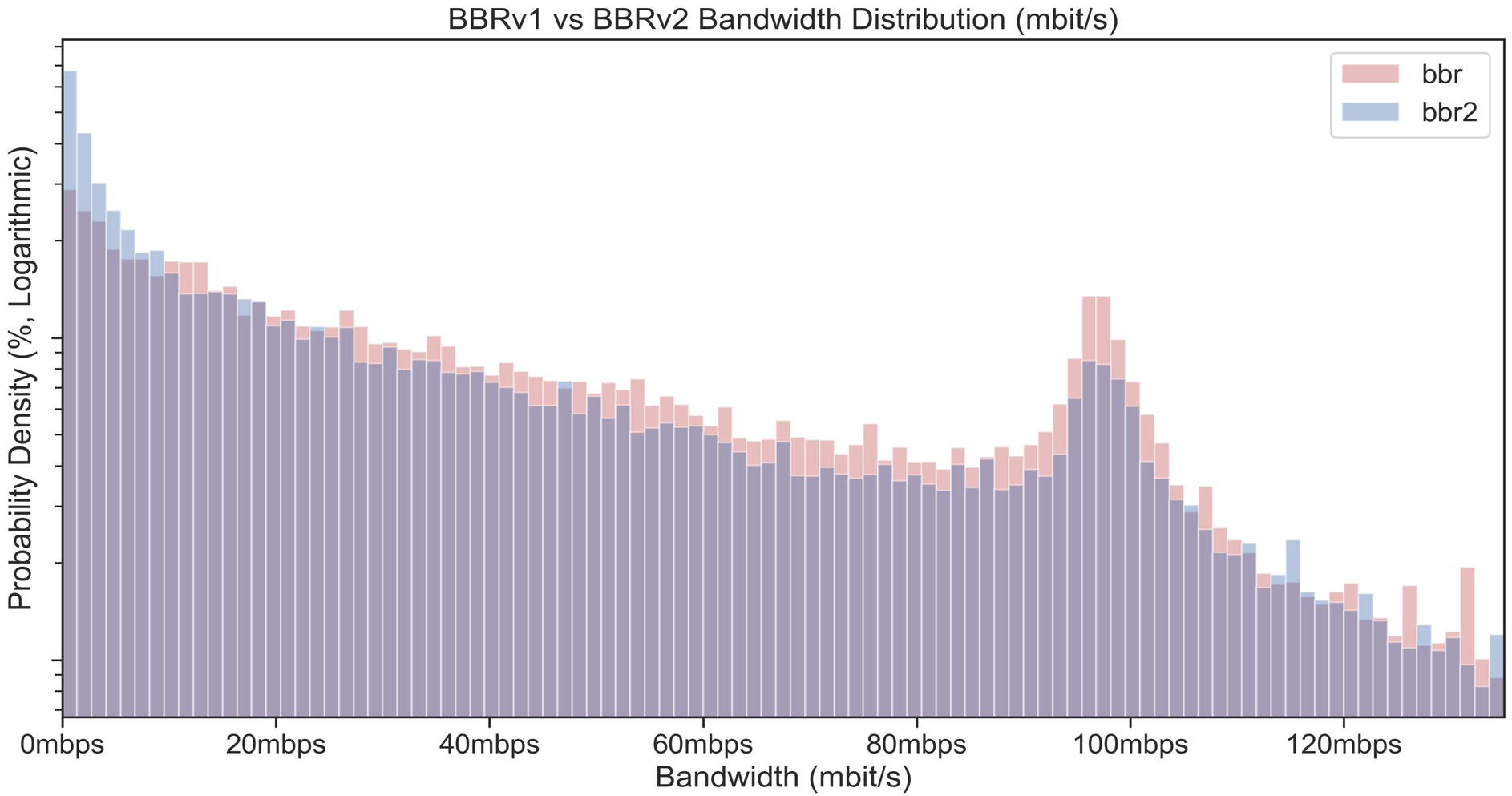


RWND-limited (vs Cubic.)

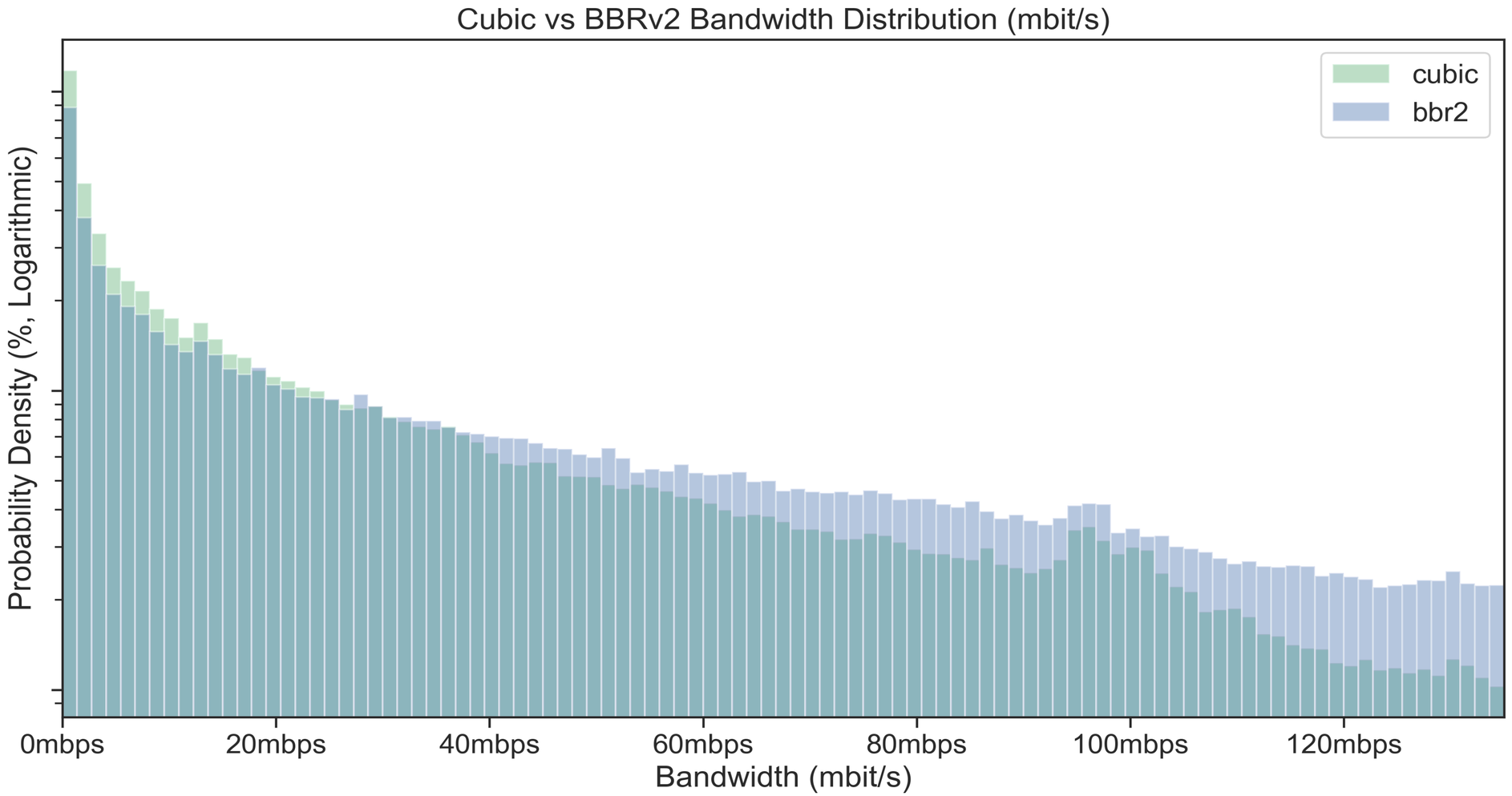


Practical Results

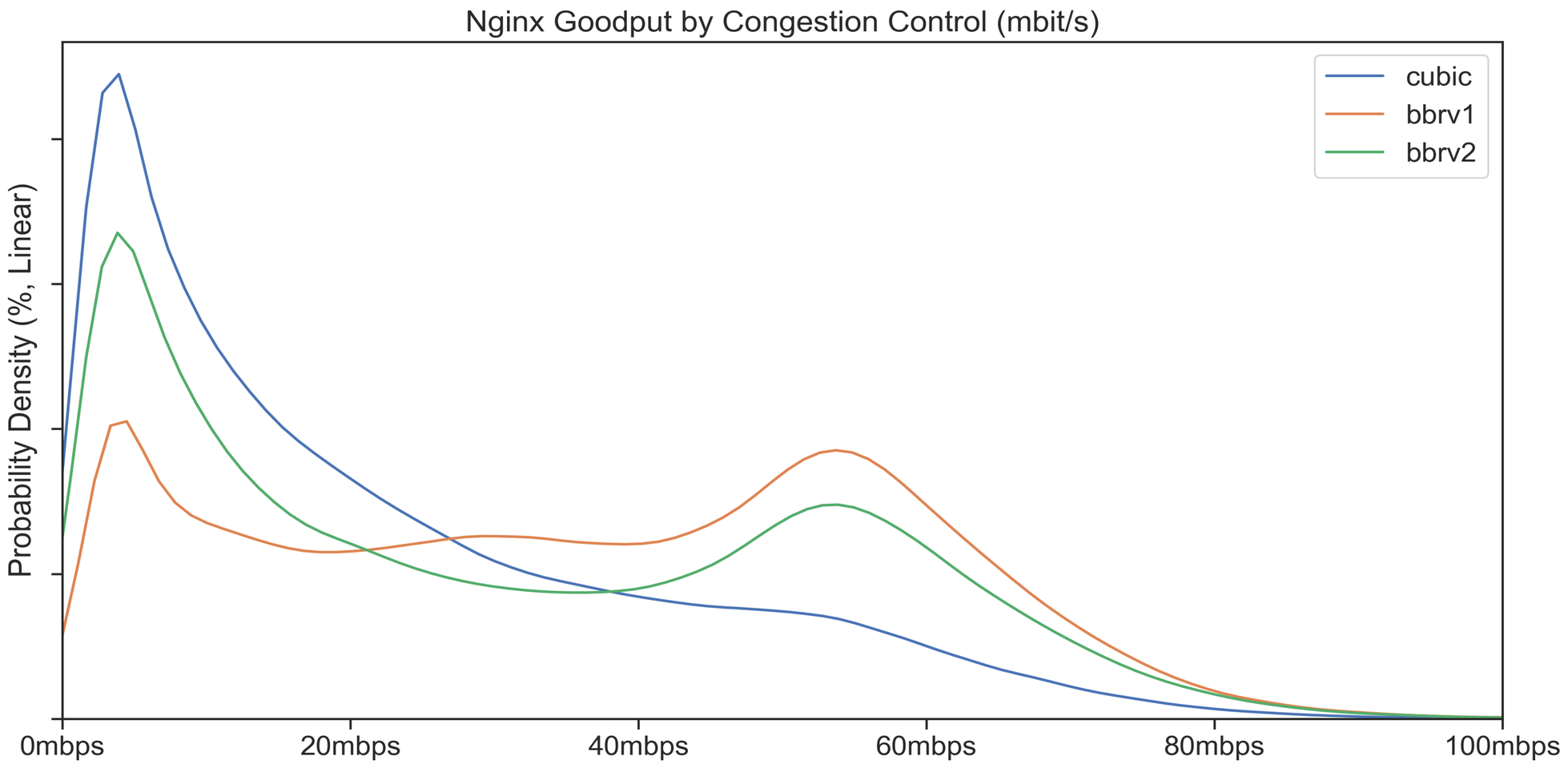
Bandwidth (vs BBRv1.)



Bandwidth (vs Cubic.)



Goodput (nginx point of view.)



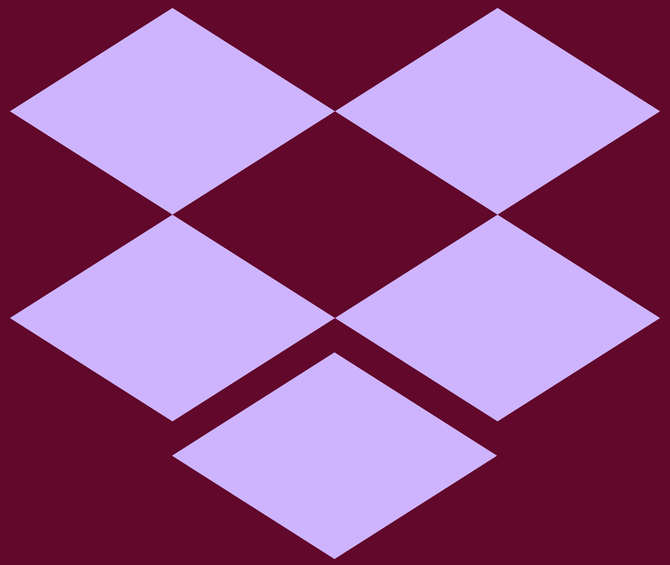
Conclusions

Issues with BBRv1.

- Low throughput for Reno/CUBIC flows sharing a bottleneck with bulk BBR flows
- Loss-agnostic; high packet loss rates if bottleneck queue < 1.5*BDP
- Low throughput for paths with high degrees of aggregation (e.g. wifi)
- Throughput variation due to low cwnd in PROBE_RTT
- ECN-agnostic

Experimental results.

- Bandwidth is comparable to CUBIC for users with lower Internet speeds.
- Bandwidth is comparable to BBRv1 for users with higher Internet speeds.
- Packet loss is 4 times lower compared to BBRv1*; still 2x higher than CUBIC.
- Data in-flight is 3 times lower compared to BBRv1; slightly lower than CUBIC.
- RTTs are lower compared to BBRv1; still higher than CUBIC.
- Higher RTT-fairness compared to BBRv1.

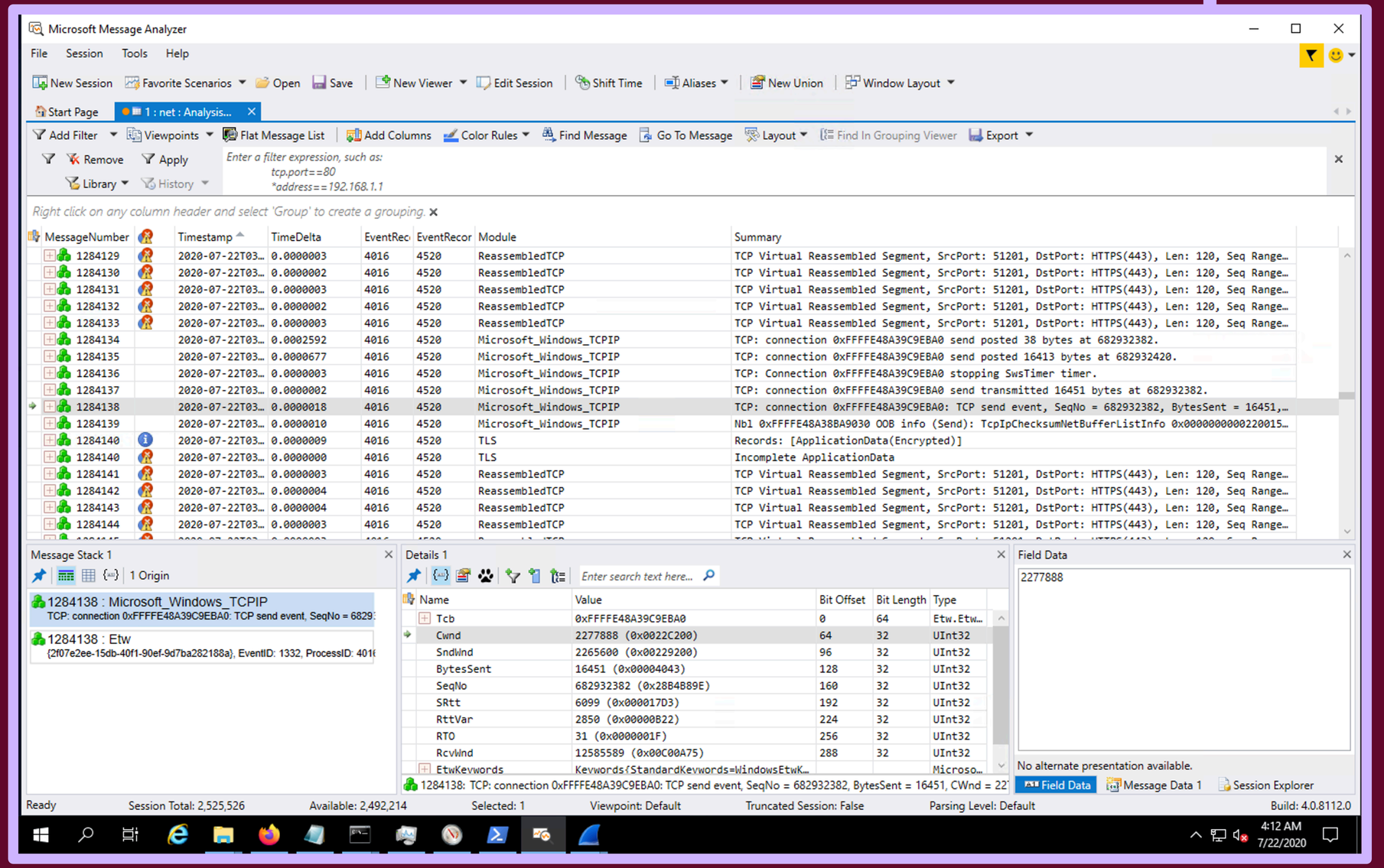


Q&A

@SaveTheRbtz

Backup Slides

Windows netsh trace vs tcpdump.



MinRTT vs Bandwidth(BBR v1&v2).

